

Tugas 13: Praktikum Mandiri 13 Machine Learning

Hisyam Wildan Alfath - 0110222206

Teknik Informatika, STT Terpadu Nurul Fikri, Depok

E-mail: hisy22206ti@student.nurulfikri.ac.id

Abstract. Praktikum ini bertujuan untuk menerapkan metode Artificial Neural Network dalam melakukan klasifikasi citra angka tulisan tangan. Dataset diproses melalui tahapan preprocessing meliputi pembacaan data, normalisasi citra, dan pembagian data menjadi data latih dan data uji. Model neural network dibangun menggunakan beberapa dense layer dan dilatih untuk mengenali pola angka 0 hingga 9. Hasil pengujian menunjukkan bahwa model mampu mencapai tingkat akurasi sekitar 85–86% pada data uji. Dengan demikian, model yang dikembangkan dapat digunakan secara efektif untuk melakukan klasifikasi angka tulisan tangan.

1. Praktikum Mandiri 13

- Instalasi Library Kaggle

```
[29] ✓ 4s | pip install kaggle
Requirement already satisfied: kaggle in /usr/local/lib/python3.12/dist-packages (1.7.4.5)
Requirement already satisfied: bleach in /usr/local/lib/python3.12/dist-packages (from kaggle) (6.3.0)
Requirement already satisfied: certifi=14.05.14 in /usr/local/lib/python3.12/dist-packages (from kaggle) (2025.11.12)
Requirement already satisfied: charset-normalizer in /usr/local/lib/python3.12/dist-packages (from kaggle) (3.4.4)
Requirement already satisfied: idna in /usr/local/lib/python3.12/dist-packages (from kaggle) (3.11)
Requirement already satisfied: protobuf in /usr/local/lib/python3.12/dist-packages (from kaggle) (5.29.5)
Requirement already satisfied: python-dateutil=2.5.3 in /usr/local/lib/python3.12/dist-packages (from kaggle) (2.9.0.post0)
Requirement already satisfied: python-slugify in /usr/local/lib/python3.12/dist-packages (from kaggle) (8.0.4)
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from kaggle) (2.32.4)
Requirement already satisfied: setuptools=21.0.0 in /usr/local/lib/python3.12/dist-packages (from kaggle) (75.2.0)
Requirement already satisfied: six=1.10 in /usr/local/lib/python3.12/dist-packages (from kaggle) (1.17.0)
Requirement already satisfied: text-unidecode in /usr/local/lib/python3.12/dist-packages (from kaggle) (1.3)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from kaggle) (4.67.1)
Requirement already satisfied: urllib3=1.15.1 in /usr/local/lib/python3.12/dist-packages (from kaggle) (2.5.0)
Requirement already satisfied: webencodings in /usr/local/lib/python3.12/dist-packages (from kaggle) (0.5.1)
```

Pada tahap ini dilakukan instalasi library Kaggle menggunakan perintah `pip install kaggle`. Setelah perintah dijalankan, sistem melakukan pengecekan terhadap library dan dependensi yang dibutuhkan. Output yang muncul menunjukkan pesan “Requirement already satisfied”, yang menandakan bahwa library Kaggle beserta seluruh dependensinya sudah terpasang sebelumnya pada environment Python. Dengan demikian, proses instalasi tidak dilakukan ulang dan library Kaggle sudah siap digunakan untuk keperluan seperti mengakses Kaggle API, mengunduh dataset, maupun menjalankan kompetisi Kaggle melalui kode Python.

- Konfigurasi Autentikasi Kaggle API

```
[30] kaggle = '/content/kaggle.json'
✓ 0s

[31] !mkdir -p ~/.kaggle
✓ 0s
    !cp {kaggle} ~/.kaggle/
    !chmod 600 ~/.kaggle/kaggle.json
```

Pada tahap ini dilakukan konfigurasi autentikasi Kaggle dengan menggunakan file kaggle.json. File tersebut berisi kredensial API Kaggle yang sebelumnya sudah diunduh, kemudian disimpan dalam variabel kaggle dengan path /content/kaggle.json. Selanjutnya dibuat folder tersembunyi .kaggle menggunakan perintah mkdir -p, lalu file kaggle.json disalin ke dalam folder tersebut. Terakhir, perintah chmod 600 digunakan untuk mengatur izin akses file agar hanya dapat dibaca dan ditulis oleh pemilik, sehingga keamanan kredensial tetap terjaga. Setelah langkah ini selesai, environment sudah terautentikasi dan siap digunakan untuk mengakses Kaggle melalui API.

- Mengunduh Dataset Handwritten Digits dari Kaggle

```
[32] !kaggle datasets download -d olafkrastovski/handwritten-digits-0-9
✓ 1s

Dataset URL: https://www.kaggle.com/datasets/olafkrastovski/handwritten-digits-0-9
License(s): CC0-1.0
handwritten-digits-0-9.zip: Skipping, found more recently modified local copy (use --force to force download)
```

Pada tahap ini dilakukan proses pengunduhan dataset handwritten-digits-0-9 dari Kaggle menggunakan perintah kaggle datasets download. Perintah tersebut menargetkan dataset milik pengguna olafkrastovski dan memerlukan autentikasi Kaggle yang telah dikonfigurasi sebelumnya. Output menampilkan URL dataset beserta informasi lisensi CC0-1.0. Selain itu, sistem memberikan keterangan “Skipping, found more recently modified local copy” yang berarti file dataset dalam bentuk .zip sudah tersedia secara lokal dengan versi yang lebih baru, sehingga proses unduhan dilewati. Dataset tersebut tetap dapat digunakan tanpa perlu mengunduh ulang.

- Ekstrak File Dataset ZIP

```
[33] ✓ 3s from zipfile import ZipFile
import os
file_name = 'handwritten-digits-0-9.zip'

extract_folder = 'dataset'
os.makedirs(extract_folder, exist_ok=True)

with ZipFile(file_name, 'r') as zip:
    zip.extractall(extract_folder)
    print('Done')
```

Done

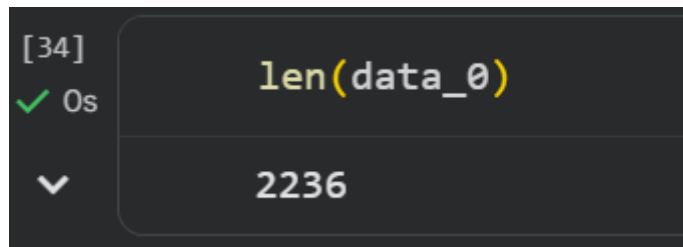
Pada tahap ini dilakukan proses ekstraksi file dataset yang masih berbentuk arsip ZIP menggunakan modul zipfile dan os. File handwritten-digits-0-9.zip ditentukan sebagai sumber data, kemudian dibuat folder tujuan bernama dataset menggunakan os.makedirs() untuk menyimpan hasil ekstraksi. Selanjutnya, file ZIP dibuka dalam mode baca dan seluruh isinya diekstrak ke dalam folder tersebut menggunakan extractall(). Output berupa teks “Done” menandakan bahwa proses ekstraksi telah berhasil dan dataset siap digunakan untuk tahap pengolahan data selanjutnya.

- Membaca Struktur Folder Dataset Angka

```
[46] ✓ 0s data_0 = os.listdir('/content/dataset/0')
data_1 = os.listdir('/content/dataset/1')
data_2 = os.listdir('/content/dataset/2')
data_3 = os.listdir('/content/dataset/3')
data_4 = os.listdir('/content/dataset/4')
data_5 = os.listdir('/content/dataset/5')
data_6 = os.listdir('/content/dataset/6')
data_7 = os.listdir('/content/dataset/7')
data_8 = os.listdir('/content/dataset/8')
data_9 = os.listdir('/content/dataset/9')
```

Pada tahap ini dilakukan pembacaan isi direktori dataset hasil ekstraksi menggunakan fungsi os.listdir(). Setiap folder merepresentasikan label angka dari 0 hingga 9, di mana masing-masing folder berisi file citra tulisan tangan sesuai dengan kelas angkanya. Variabel data_0 hingga data_9 digunakan untuk menyimpan daftar nama file pada setiap folder tersebut. Langkah ini bertujuan untuk memastikan bahwa struktur dataset sudah terbaca dengan benar dan seluruh data gambar untuk masing-masing kelas tersedia sebelum masuk ke tahap pemrosesan dan pelatihan model.

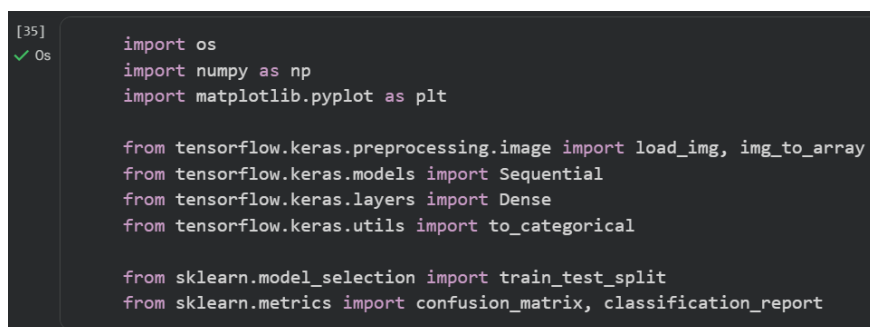
- Mengetahui Jumlah Data pada Kelas Angka 0



```
[34] len(data_0)
✓ Os 2236
```

Pada tahap ini dilakukan perhitungan jumlah data pada folder kelas angka 0 menggunakan fungsi `len()`. Variabel `data_0` berisi daftar nama file gambar yang sebelumnya diperoleh dari hasil pembacaan direktori. Output menunjukkan nilai 2236, yang berarti terdapat 2236 file citra tulisan tangan untuk kelas angka 0. Informasi ini digunakan untuk memahami distribusi jumlah data pada setiap kelas sebelum dilakukan analisis lebih lanjut atau proses pelatihan model.

- Menampilkan Daftar Nama Kolom Dataset



```
[35] import os
import numpy as np
import matplotlib.pyplot as plt

from tensorflow.keras.preprocessing.image import load_img, img_to_array
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.utils import to_categorical

from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, classification_report
```

Pada tahap ini dilakukan proses import berbagai library yang dibutuhkan untuk pengolahan data, visualisasi, serta pembuatan dan evaluasi model klasifikasi citra. Library `os` digunakan untuk mengelola direktori dan file, sedangkan `numpy` digunakan untuk operasi numerik pada data array. `matplotlib.pyplot` digunakan untuk visualisasi data dan hasil analisis. Selain itu, beberapa modul dari `tensorflow.keras` diimpor untuk keperluan pemrosesan citra dan pembangunan model jaringan saraf, seperti `load_img` dan `img_to_array` untuk membaca dan mengonversi gambar, `Sequential` dan `Dense` untuk membangun model, serta `to_categorical` untuk mengubah label menjadi format kategorikal. Library dari `sklearn` digunakan untuk membagi data menjadi data latih dan uji serta mengevaluasi performa model melalui `confusion matrix` dan `classification report`. Langkah ini menjadi dasar sebelum masuk ke tahap preprocessing data dan pelatihan model.

- Pemeriksaan Missing Value pada Dataset

```
[36] ✓ 8s
dataset_path = "/content/dataset"
image_size = (28, 28)

X = []
y = []

AUTO_INVERT = True

for label in range(10):
    folder_path = os.path.join(dataset_path, str(label))
    for file in os.listdir(folder_path):

        if not file.lower().endswith((".png", ".jpg", ".jpeg", ".bmp")):
            continue

        img_path = os.path.join(folder_path, file)

        img = load_img(img_path, color_mode="grayscale", target_size=image_size)
        img_array = img_to_array(img).astype("float32")

        img_array = img_array / 255.0

        if AUTO_INVERT and img_array.mean() > 0.5:
            img_array = 1.0 - img_array

        X.append(img_array)
        y.append(label)

X = np.array(X, dtype=np.float32)
y = np.array(y, dtype=np.int64)
print("X shape:", X.shape)
print("y shape:", y.shape)
print("X min/max/mean:", X.min(), X.max(), X.mean())

X shape: (21555, 28, 28, 1)
y shape: (21555,)
X min/max/mean: 0.0 1.0 0.050949883
```

Pada tahap ini dilakukan proses preprocessing dataset citra tulisan tangan yang berada pada direktori /content/dataset. Setiap gambar dibaca dari folder label angka 0 hingga 9, kemudian diubah menjadi citra grayscale dengan ukuran 28×28 piksel. Gambar yang telah dibaca dikonversi ke dalam bentuk array numerik dan dinormalisasi dengan membagi nilai piksel dengan 255 agar berada pada rentang 0 hingga 1. Selain itu, diterapkan proses auto invert untuk membalik warna citra secara otomatis apabila rata-rata nilai piksel terlalu terang, sehingga format citra menjadi lebih konsisten. Seluruh data citra disimpan ke dalam variabel X, sedangkan label kelasnya disimpan pada variabel y. Output menunjukkan bahwa dataset berhasil dibentuk dengan total 21.555 data citra berukuran (28, 28, 1) dan label yang sesuai, serta nilai minimum, maksimum, dan rata-rata piksel yang menandakan proses normalisasi berjalan dengan baik.

- Transformasi Citra ke Bentuk Vektor (Flattening)

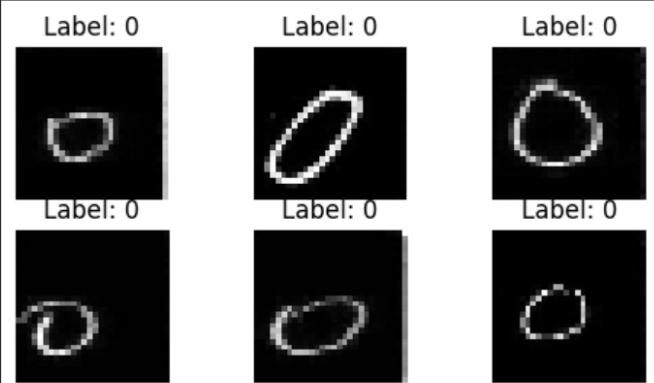
```
[37] X = X.reshape(X.shape[0], 28 * 28)
      print("After flatten:", X.shape)

After flatten: (21555, 784)
```

Pada tahap ini dilakukan proses flattening pada data citra yang sebelumnya berbentuk matriks dua dimensi berukuran 28×28 piksel. Setiap citra diubah menjadi satu vektor satu dimensi dengan panjang 784 (28×28) menggunakan fungsi reshape(). Proses ini diperlukan agar data citra dapat digunakan sebagai input pada model jaringan saraf berbasis fully connected layer. Output menunjukkan perubahan bentuk data menjadi (21555, 784), yang berarti terdapat 21.555 data citra dan masing-masing direpresentasikan oleh 784 fitur.

- Visualisasi Sampel Citra dan Label Dataset

```
[38] plt.figure(figsize=(6,3))
      for i in range(6):
          plt.subplot(2, 3, i+1)
          plt.imshow(X[i].reshape(28,28), cmap="gray")
          plt.title(f"Label: {y[i]}")
          plt.axis("off")
      plt.show()
```



Pada tahap ini dilakukan visualisasi beberapa sampel citra dari dataset untuk memastikan bahwa data telah diproses dengan benar. Sebanyak enam gambar pertama dari variabel X ditampilkan dalam bentuk grid 2×3 menggunakan matplotlib. Setiap citra yang sebelumnya telah di-flatten dikembalikan ke ukuran asli 28×28 piksel agar dapat ditampilkan sebagai gambar grayscale. Judul pada masing-masing gambar menampilkan label kelas yang tersimpan pada variabel y, sehingga dapat diverifikasi

kesesuaian antara citra dan labelnya. Visualisasi ini menunjukkan bahwa data citra dan label telah sesuai dan siap digunakan pada tahap pelatihan model.

- Pembagian Data Menjadi Data Latih dan Data Uji

```
[39]
✓ Os      X_train, X_test, y_train_raw, y_test_raw = train_test_split(
          X, y,
          test_size=0.2,
          random_state=42,
          stratify=y
          )
```

Pada tahap ini dataset dibagi menjadi data latih dan data uji menggunakan fungsi `train_test_split`. Sebanyak 80% data digunakan sebagai data latih dan 20% sisanya sebagai data uji. Parameter `random_state=42` digunakan agar proses pembagian data bersifat konsisten dan dapat direproduksi. Selain itu, parameter `stratify=y` memastikan bahwa proporsi setiap kelas angka pada data latih dan data uji tetap seimbang. Hasil dari proses ini adalah terbentuknya variabel `X_train`, `X_test`, `y_train_raw`, dan `y_test_raw` yang akan digunakan pada tahap pelatihan dan evaluasi model.

- Konversi Label ke Format One-Hot Encoding

```
[40]
✓ Os      y_train = to_categorical(y_train_raw, num_classes=10)
          y_test = to_categorical(y_test_raw, num_classes=10)

          print("y_train:", y_train.shape, "y_test:", y_test.shape)

          y_train: (17244, 10) y_test: (4311, 10)
```

Pada tahap ini label kelas pada data latih dan data uji dikonversi ke dalam format one-hot encoding menggunakan fungsi `to_categorical`. Label numerik dari 0 hingga 9 diubah menjadi vektor dengan panjang 10, sesuai dengan jumlah kelas yang ada. Proses ini diperlukan agar label dapat digunakan dalam pelatihan model klasifikasi multikelas menggunakan neural network. Output menunjukkan bahwa data latih memiliki bentuk (17244, 10) dan data uji (4311, 10), yang menandakan bahwa setiap data kini memiliki representasi label kategorikal yang siap digunakan pada tahap pelatihan dan evaluasi model.

- Perancangan dan Kompilasi Model Neural Network

```
[41] ✓ 0s
model = Sequential([
    Dense(128, activation="relu", input_shape=(784,)),
    Dense(64, activation="relu"),
    Dense(10, activation="softmax")
])

model.compile(
    optimizer="adam",
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)

model.summary()
```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarning: Do not pass an `input\_shape` / `input\_dim` argument to a layer.
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential_1"

Layer (type)	Output Shape	Param #
dense_3 (Dense)	(None, 128)	109,488
dense_4 (Dense)	(None, 64)	8,256
dense_5 (Dense)	(None, 10)	650

Total params: 109,394 (427.29 KB)
Trainable params: 109,394 (427.29 KB)
Non-trainable params: 0 (0.00 B)

Bagian ini menunjukkan proses perancangan arsitektur model Artificial Neural Network menggunakan pendekatan Sequential. Model dibangun dengan tiga fully connected layer, yaitu layer pertama dengan 128 neuron dan fungsi aktivasi ReLU sebagai penerima input berukuran 784 fitur, layer kedua dengan 64 neuron dan aktivasi ReLU, serta layer output dengan 10 neuron dan fungsi aktivasi Softmax untuk melakukan klasifikasi multikelas angka 0–9.

Setelah arsitektur ditentukan, model dikompilasi menggunakan optimizer Adam dan fungsi loss categorical crossentropy, yang sesuai untuk permasalahan klasifikasi multikelas. Metrik accuracy digunakan untuk mengukur performa model selama pelatihan. Output `model.summary()` menampilkan struktur model secara detail, termasuk jumlah parameter pada setiap layer dan total parameter sebanyak 109.386, yang seluruhnya bersifat trainable dan siap digunakan pada proses pelatihan model.

- Proses Pelatihan Model Neural Network

```
[42] history = model.fit(  
    X_train, y_train,  
    epochs=20,  
    batch_size=128,  
    validation_split=0.1,  
    verbose=1  
)  
  
Epoch 1/20  
122/122 — 3s 14ms/step - accuracy: 0.3310 - loss: 2.0085 - val_accuracy: 0.5467 - val_loss: 1.4200  
Epoch 2/20  
122/122 — 0s 3ms/step - accuracy: 0.6032 - loss: 1.2983 - val_accuracy: 0.6377 - val_loss: 1.1438  
Epoch 3/20  
122/122 — 0s 3ms/step - accuracy: 0.7088 - loss: 0.9806 - val_accuracy: 0.7229 - val_loss: 0.9047  
Epoch 4/20  
122/122 — 0s 3ms/step - accuracy: 0.7879 - loss: 0.7266 - val_accuracy: 0.7786 - val_loss: 0.7437  
Epoch 5/20  
122/122 — 0s 3ms/step - accuracy: 0.8506 - loss: 0.5472 - val_accuracy: 0.8012 - val_loss: 0.6421  
Epoch 6/20  
122/122 — 0s 3ms/step - accuracy: 0.8827 - loss: 0.4306 - val_accuracy: 0.8307 - val_loss: 0.5633  
Epoch 7/20  
122/122 — 0s 3ms/step - accuracy: 0.9155 - loss: 0.3285 - val_accuracy: 0.8423 - val_loss: 0.5319  
Epoch 8/20  
122/122 — 1s 4ms/step - accuracy: 0.9296 - loss: 0.2715 - val_accuracy: 0.8417 - val_loss: 0.5195  
Epoch 9/20  
122/122 — 1s 4ms/step - accuracy: 0.9461 - loss: 0.2243 - val_accuracy: 0.8586 - val_loss: 0.4921  
Epoch 10/20  
122/122 — 1s 4ms/step - accuracy: 0.9546 - loss: 0.1912 - val_accuracy: 0.8580 - val_loss: 0.4730
```

Bagian ini menampilkan proses pelatihan model neural network menggunakan data latih yang telah disiapkan sebelumnya. Model dilatih selama 20 epoch dengan ukuran batch sebesar 128, serta menggunakan 10% dari data latih sebagai data validasi. Parameter verbose=1 digunakan agar proses pelatihan ditampilkan secara rinci pada setiap epoch.

Output menunjukkan perkembangan nilai accuracy dan loss baik pada data latih maupun data validasi. Terlihat bahwa akurasi model meningkat secara bertahap dari epoch ke epoch, sementara nilai loss menurun, yang menandakan bahwa model berhasil mempelajari pola dari data. Nilai validation accuracy juga mengalami peningkatan, sehingga dapat disimpulkan bahwa model memiliki kemampuan generalisasi yang cukup baik dan siap untuk dievaluasi menggunakan data uji.

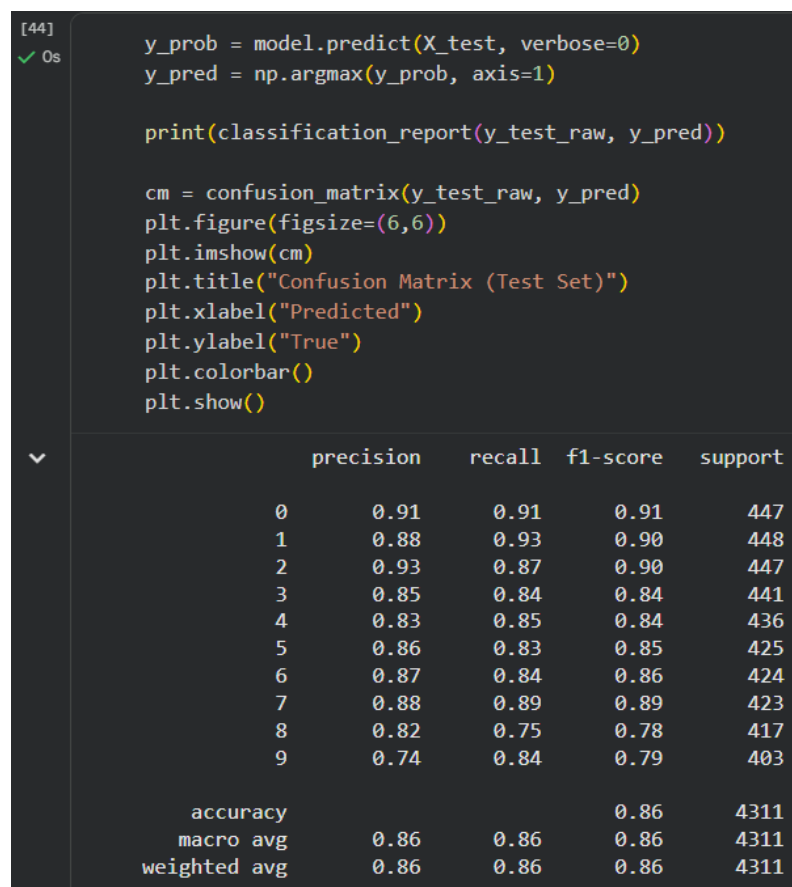
- Evaluasi Performa Model Menggunakan Data Uji

```
[43] test_loss, test_acc = model.evaluate(X_test, y_test, verbose=0)  
print("Test Loss      :", test_loss)  
print("Test Accuracy  :", test_acc)  
  
Test Loss      : 0.570957601070404  
Test Accuracy  : 0.8575736284255981
```

Bagian ini menunjukkan proses evaluasi model yang telah dilatih dengan menggunakan data uji (X_test dan y_test). Fungsi model.evaluate() digunakan untuk menghitung nilai loss dan accuracy tanpa menampilkan proses secara rinci (verbose=0).

Output yang dihasilkan menunjukkan nilai Test Loss sebesar 0.57 dan Test Accuracy sekitar 85.76%. Hasil ini menandakan bahwa model mampu melakukan klasifikasi angka tulisan tangan dengan tingkat akurasi yang cukup baik pada data yang belum pernah dilihat sebelumnya, sehingga model dapat dikatakan memiliki performa yang stabil dan dapat diandalkan.

- Analisis Hasil Prediksi dan Confusion Matrix

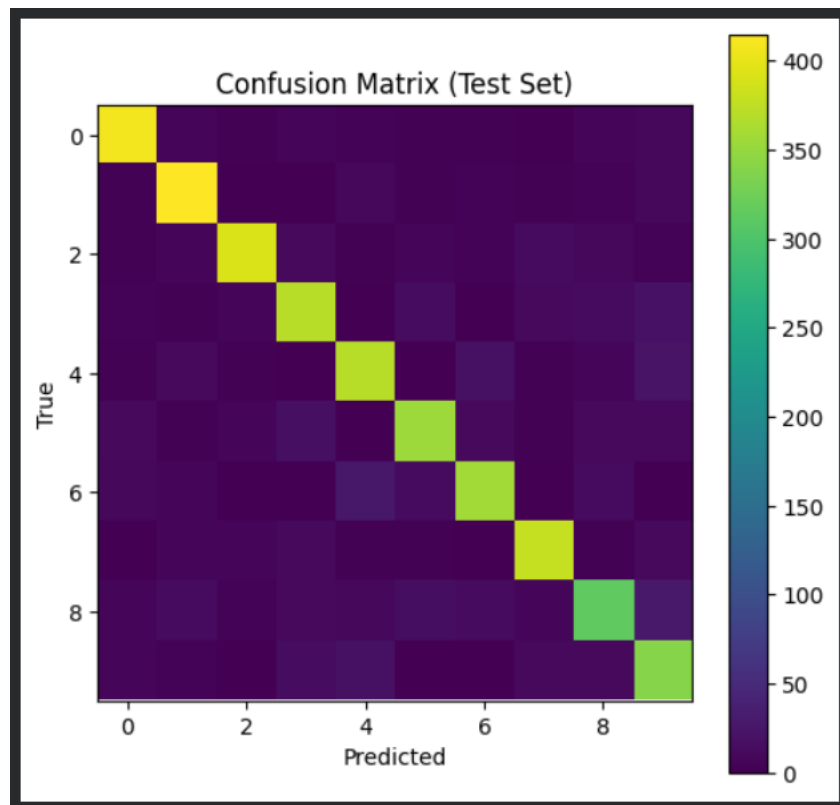


Bagian ini menampilkan proses analisis hasil prediksi model pada data uji. Model menghasilkan probabilitas prediksi untuk setiap kelas menggunakan `model.predict`, kemudian kelas dengan probabilitas tertinggi dipilih sebagai hasil prediksi akhir. Hasil tersebut dibandingkan dengan label asli untuk menghasilkan classification report yang berisi nilai precision, recall, f1-score, dan support untuk setiap kelas angka 0 hingga 9.

Selain itu, dibuat confusion matrix untuk memvisualisasikan hubungan antara label sebenarnya dan label hasil prediksi. Visualisasi ini membantu melihat kelas mana yang paling sering diklasifikasikan dengan benar maupun yang masih mengalami

kesalahan. Berdasarkan laporan klasifikasi, model mencapai akurasi sekitar 86% dengan performa yang relatif seimbang di hampir seluruh kelas, meskipun terdapat beberapa kelas yang masih memiliki nilai recall lebih rendah. Hasil ini menunjukkan bahwa model sudah bekerja dengan baik dan memberikan gambaran jelas mengenai kekuatan serta kelemahan model dalam melakukan klasifikasi.

- Interpretasi Visual Confusion Matrix pada Data Uji



Gambar tersebut menampilkan confusion matrix hasil pengujian model klasifikasi angka tulisan tangan pada data uji. Sumbu vertikal menunjukkan label sebenarnya (True), sedangkan sumbu horizontal menunjukkan label hasil prediksi model (Predicted). Warna yang semakin terang menandakan jumlah prediksi yang semakin besar pada kombinasi kelas tertentu. Terlihat bahwa sebagian besar nilai tertinggi berada pada diagonal utama, yang menunjukkan bahwa model mampu mengklasifikasikan sebagian besar data dengan benar untuk setiap kelas angka. Namun, masih terdapat beberapa sel di luar diagonal yang menandakan adanya kesalahan prediksi, yaitu ketika suatu angka diprediksi sebagai angka lain. Visualisasi ini membantu dalam memahami pola kesalahan model secara lebih intuitif dan menjadi

dasar untuk evaluasi lebih lanjut serta kemungkinan peningkatan performa model di tahap berikutnya.

- Pemilihan Kolom Numerik untuk Uji Normalitas

```
[45]
✓ Os
import matplotlib.pyplot as plt

plt.figure(figsize=(12,4))

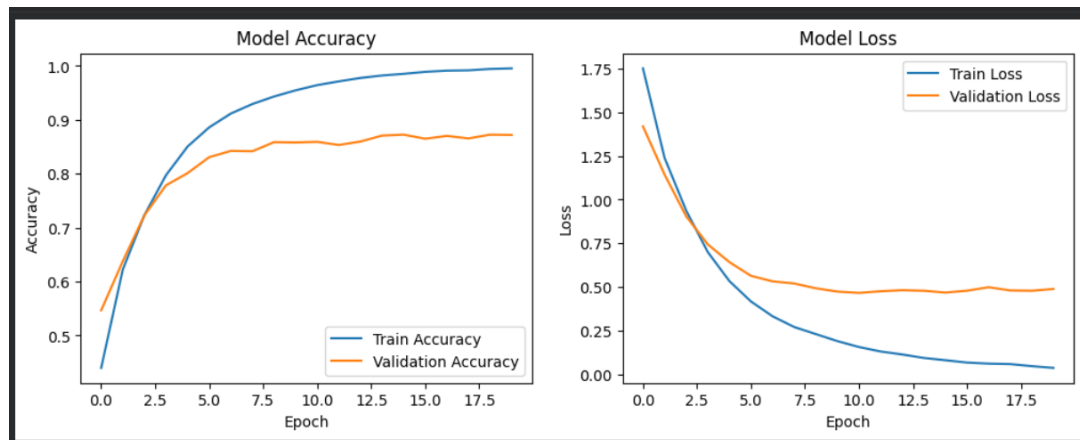
# Accuracy
plt.subplot(1,2,1)
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Model Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()

# Loss
plt.subplot(1,2,2)
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Model Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()

plt.show()
```

Bagian ini digunakan untuk memvisualisasikan kinerja model selama proses pelatihan dalam bentuk grafik. Grafik pertama menampilkan perbandingan antara training accuracy dan validation accuracy pada setiap epoch, sehingga dapat diamati bagaimana kemampuan model dalam mempelajari data latih sekaligus melakukan generalisasi terhadap data validasi. Grafik kedua menunjukkan perubahan nilai training loss dan validation loss pada setiap epoch, yang berguna untuk melihat apakah proses pembelajaran berjalan dengan baik atau terjadi overfitting.

- Analisis Grafik Akurasi dan Loss Model



Grafik pertama menunjukkan perkembangan akurasi model selama proses pelatihan. Terlihat bahwa training accuracy meningkat secara konsisten hingga mendekati nilai maksimum, sedangkan validation accuracy juga mengalami peningkatan dan cenderung stabil di kisaran tertentu. Perbedaan antara kedua kurva masih dalam batas wajar, yang menandakan bahwa model mampu belajar dengan baik tanpa mengalami overfitting yang berlebihan.

Grafik kedua memperlihatkan perubahan nilai loss pada data latih dan data validasi. Training loss menurun secara signifikan seiring bertambahnya epoch, sementara validation loss juga menurun di awal pelatihan dan kemudian relatif stabil. Pola ini menunjukkan bahwa proses pembelajaran berlangsung dengan baik dan model telah mencapai kondisi konvergen. Secara keseluruhan, grafik ini mengonfirmasi bahwa model memiliki performa yang stabil dan layak digunakan untuk klasifikasi angka tulisan tangan.

- Kesimpulan

Praktikum mandiri ini berhasil menerapkan proses klasifikasi citra angka tulisan tangan menggunakan neural network. Dataset diproses dengan baik melalui tahap preprocessing, normalisasi, dan pembagian data latih serta data uji. Model yang dibangun mampu mempelajari pola data secara efektif dengan peningkatan akurasi yang konsisten selama pelatihan. Hasil evaluasi menunjukkan akurasi sekitar 85–86% pada data uji dengan performa yang relatif seimbang di setiap kelas. Secara keseluruhan, model menunjukkan kinerja yang stabil dan layak digunakan untuk tugas klasifikasi angka.

Link Praktikum 13:

https://github.com/HisyamWildan/TI03_HisyamW.A_0110222206/blob/main/Pertemuan%2013/Praktikum%20dan%20Tugas%20Mandiri%2013/notebooks/praktikum13.ipynb

Tugas Mandiri 13:

https://github.com/HisyamWildan/TI03_HisyamW.A_0110222206/blob/main/Pertemuan%2013/Praktikum%20dan%20Tugas%20Mandiri%2013/notebooks/latihan13.ipynb